

Magnetic Collector

Collecte par champ d'attraction

ATAP

Informations

Nom : Richard

Prénom : Nathan

Classe : 3APROG

Date : _____

Numéro de disque : _____

Commentaires :

Table des matières

1 Contexte	2
2 Mandat	2
2.1 Sujet	2
2.2 Objectif	2
3 Fonctionnalités attendues	2
4 Contraintes techniques	3
5 Conception	3
6 Développement et organisation	4
7 Tests et validation	4
8 Résultat attendu	5
9 Améliorations facultatives	5
10 Documentation	5
11 Ressources externes et assistance	6
12 Livrables et présentation finale	6
12.1 Livrables	6
12.2 Présentation finale	6
13 Axes d'évaluation	7

1 Contexte

Dans le cadre de votre apprentissage de **C# et MonoGame**, vous devez réaliser individuellement une application interactive 2D à partir du mandat qui vous est attribué.

Ce projet doit vous permettre de mobiliser les notions étudiées durant les différents travaux pratiques : boucle de jeu, gestion du temps, entrées, éléments graphiques 2D, déplacements ou évolutions temporelles, interactions, collections d'objets et organisation du code.

Le sujet et les fonctionnalités principales sont imposés.



Vous restez cependant responsable de la **conception de votre solution**, de l'organisation de votre code et des choix techniques nécessaires à sa réalisation.

2 Mandat

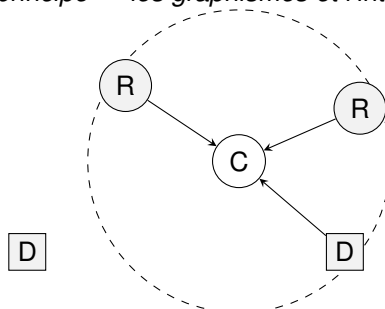
2.1 Sujet

Magnetic Collector

Collecte par champ d'attraction

Vous pilotez un collecteur spatial capable de produire temporairement un champ magnétique. Des ressources et des débris traversent la zone. Le champ attire simultanément les objets suffisamment proches du collecteur.

Schéma de principe — les graphismes et l'interface restent libres.



Lorsque le champ est actif, les objets situés dans son rayon se déplacent vers le collecteur, y compris les débris dangereux.

2.2 Objectif

Récupérer la quantité demandée de ressources sans perdre toutes les vies. L'utilisation du champ doit permettre d'attirer les ressources tout en évitant d'amener les débris dangereux jusqu'au collecteur.

3 Fonctionnalités attendues

Votre application doit au minimum proposer les fonctionnalités suivantes :

- ☐ Le joueur peut déplacer le collecteur dans la zone.
- ☐ Des ressources et des débris apparaissent progressivement.
- ☐ Plusieurs objets peuvent être présents simultanément.
- ☐ Le joueur peut activer un champ magnétique pendant une durée limitée.

- ☐ Lorsque le champ est actif, les objets situés dans son rayon se déplacent progressivement vers le collecteur.
- ☐ Plusieurs objets peuvent être influencés en même temps.
- ☐ Une ressource atteignant le collecteur est supprimée et augmente la progression.
- ☐ Un débris dangereux atteignant le collecteur fait perdre une vie et disparaît.
- ☐ Les objets ayant définitivement quitté la zone sont supprimés.
- ☐ Le champ ne peut pas rester actif en permanence et doit redevenir disponible après un délai ou une recharge.
- ☐ La fréquence d'apparition augmente progressivement.
- ☐ La progression, les vies et la disponibilité du champ sont visibles.
- ☐ La partie est gagnée lorsque l'objectif de ressources est atteint et perdue lorsque les vies sont épuisées.
- ☐ Le joueur peut recommencer une partie terminée.

Les fonctionnalités indiquées dans cette section constituent le **socle obligatoire du projet**.
Les améliorations personnelles ne doivent être entreprises qu'une fois ce socle fonctionnel.



4 Contraintes techniques

Les contraintes suivantes sont communes à tous les projets :

- ☐ l'application doit être développée en **C# avec MonoGame** ;
- ☐ le projet doit exploiter correctement la boucle `Update()` / `Draw()` ;
- ☐ les déplacements et comportements temporels doivent utiliser `GameTime` ;
- ☐ le programme doit être organisé en plusieurs classes cohérentes ;
- ☐ les collections doivent être utilisées lorsqu'elles sont pertinentes ;
- ☐ les collisions ou interactions doivent être gérées par le programme ;
- ☐ le code doit rester lisible, structuré et maintenable ;
- ☐ le projet doit être versionné avec Git.

Les contraintes suivantes sont propres à votre sujet :

- ☐ Le joueur commence avec 3 vies.
- ☐ Au moins 2 catégories d'objets sont utilisées : ressources et débris dangereux.
- ☐ Au moins 6 objets doivent pouvoir être présents simultanément.
- ☐ Le champ possède un rayon limité et une durée d'activation limitée.
- ☐ L'objectif minimal correspond à 25 ressources collectées.

5 Conception

Avant de développer l'ensemble de l'application, vous devez réfléchir à l'organisation de votre solution.

La conception doit notamment permettre d'identifier :

- ☐ les principales classes nécessaires ;
- ☐ leurs responsabilités ;
- ☐ les relations importantes entre les objets ;

- ☐ les principales données manipulées ;
- ☐ les mécanismes nécessaires au fonctionnement de l'application.

Un **diagramme de classes initial** doit être réalisé avant ou au début du développement.

Ce diagramme constitue une première proposition de conception. Il peut évoluer en fonction des besoins identifiés pendant la réalisation.

La documentation finale devra présenter le **diagramme de classes correspondant à la solution réellement développée**.

6 Développement et organisation

Vous êtes responsable de l'organisation de votre travail et de la progression de votre projet.

Vous devez notamment prévoir du temps pour :

- ☐ analyser le mandat et concevoir votre solution ;
- ☐ développer progressivement les fonctionnalités obligatoires ;
- ☐ tester et corriger régulièrement votre application ;
- ☐ améliorer la qualité de votre code ;
- ☐ finaliser la documentation ;
- ☐ préparer la présentation finale.

Le développement doit respecter les principes étudiés durant la formation.

Une attention particulière sera portée :

- ☐ à la séparation des responsabilités ;
- ☐ à la pertinence des classes créées ;
- ☐ au nommage des classes, méthodes, propriétés et variables ;
- ☐ à la lisibilité et à la structure du code ;
- ☐ à l'absence de duplication inutile ;
- ☐ à la gestion correcte des collections et des objets ;
- ☐ à la maîtrise du cycle de vie des objets ;
- ☐ à la gestion correcte du temps et des déplacements.

Le dépôt Git doit montrer une **progression régulière du travail**.

Un projet simple, complet et maîtrisé est préférable à une application plus ambitieuse mais inachevée.



7 Tests et validation

Vous devez tester votre application tout au long du développement.

Les tests doivent notamment vérifier :

- ☐ le fonctionnement des commandes et interactions utilisateur ;
- ☐ les interactions et collisions entre les différents éléments ;
- ☐ les conditions particulières ou limites ;

- ☐ les conditions de victoire, de défaite ou de fin lorsqu'elles existent ;
- ☐ les changements d'état importants de l'application ;
- ☐ la stabilité générale de l'application.

Les principaux tests réalisés et les problèmes rencontrés doivent être présentés dans la documentation.

8 Résultat attendu

Le projet est considéré comme complet lorsque :

- ☐ toutes les fonctionnalités obligatoires de votre sujet sont réalisées ;
- ☐ l'application peut être exécutée et utilisée normalement ;
- ☐ les principales situations prévues ont été testées ;
- ☐ le code est organisé, lisible et compréhensible ;
- ☐ la documentation demandée est terminée ;
- ☐ le dépôt Git contient l'historique du développement.

Les éléments graphiques supplémentaires, effets visuels, sons, musiques, animations ou autres améliorations ne sont pas nécessaires, sauf lorsqu'ils font explicitement partie des fonctionnalités obligatoires de votre sujet.



9 Améliorations facultatives

Lorsque toutes les fonctionnalités obligatoires sont réalisées et fonctionnelles, vous pouvez améliorer votre application.

Pour votre sujet, les pistes suivantes peuvent notamment être envisagées :

- ☐ Ajouter plusieurs valeurs de ressources.
- ☐ Ajouter un bonus augmentant temporairement le rayon du champ.
- ☐ Ajouter un débris résistant davantage à l'attraction.
- ☐ Ajouter un multiplicateur pour plusieurs ressources capturées lors d'une même activation.

Ces propositions ne sont pas obligatoires et ne constituent pas une liste exhaustive.

Vous pouvez également proposer vos propres améliorations lorsqu'elles restent cohérentes avec le projet.

Une fonctionnalité facultative ne compense pas une fonctionnalité obligatoire absente ou non fonctionnelle.



10 Documentation

Une **documentation technique courte** accompagne le projet.

Elle doit permettre de comprendre votre solution et les principaux choix effectués sans décrire chaque ligne de code.

Elle contient au minimum :

- 1 une présentation succincte de l'application ;

- 2 le diagramme de classes final ;
- 3 une explication des principaux choix de conception ;
- 4 la présentation d'un mécanisme technique significatif du projet ;
- 5 les principaux tests réalisés ;
- 6 les difficultés rencontrées et les solutions apportées ;
- 7 un court bilan personnel.

La documentation doit rester proportionnée au projet.

11 Ressources externes et assistance

L'utilisation de documentation, de ressources externes et d'outils d'assistance est autorisée dans le cadre défini par le formateur.

Toute ressource ou assistance ayant contribué de manière significative à la réalisation du projet doit être identifiable et déclarée.

Vous devez dans tous les cas rester capable :

- ☐ d'expliquer le code remis ;
- ☐ de justifier vos choix ;
- ☐ d'expliquer les solutions obtenues avec une assistance ;
- ☐ de modifier ou corriger votre propre programme.

L'utilisation d'une ressource ou d'un outil d'assistance ne remplace pas la compréhension du travail réalisé.



12 Livrables et présentation finale

12.1 Livrables

À la fin du projet, vous devez remettre :

- ☐ le projet MonoGame complet et exécutable ;
- ☐ le dépôt Git contenant l'historique du développement ;
- ☐ la documentation technique au format PDF ;
- ☐ les ressources nécessaires au fonctionnement de l'application ;
- ☐ la déclaration des ressources et assistances utilisées.

Les modalités pratiques et la date de remise sont communiquées par le formateur.

12.2 Présentation finale

Le projet se termine par une courte présentation de votre réalisation.

Vous devez être capable :

- ☐ de présenter le fonctionnement de votre application ;
- ☐ de démontrer les principales fonctionnalités ;
- ☐ d'expliquer son architecture ;

- ☐ de présenter un élément technique significatif ;
- ☐ d'expliquer une difficulté rencontrée et sa résolution ;
- ☐ de répondre à des questions sur votre propre code.

13 Axes d'évaluation

L'évaluation porte notamment sur :

- ☐ la compréhension et le respect du mandat ;
- ☐ la qualité et la cohérence de la conception ;
- ☐ la qualité et la structure du code ;
- ☐ l'utilisation pertinente de MonoGame et de C# ;
- ☐ le fonctionnement des fonctionnalités obligatoires ;
- ☐ la qualité des tests et des corrections ;
- ☐ la progression du travail et l'utilisation de Git ;
- ☐ la qualité et la pertinence de la documentation ;
- ☐ la maîtrise du projet lors de la présentation.

La quantité de fonctionnalités réalisées n'est pas le principal objectif.

La priorité est donnée à une réalisation **fonctionnelle, structurée, testée et maîtrisée**.



La rédaction de ce document repose principalement sur le travail de l'auteur, complété par la consultation de ressources spécialisées et l'usage raisonné d'un outil d'intelligence artificielle pour en améliorer la structure et la lisibilité. Les informations présentées ont été vérifiées et validées.